

Intermediate (swap in a structured parser) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
# structured_output_practice.py – Intermediate section
from pydantic import BaseModel
from langchain_openai import ChatOpenAI

class Person(BaseModel):
    name: str
    age: int

structured_model = ChatOpenAI().with_structured_output(Person)

good_result = structured_model.invoke("Extract: Maria is 34 years old.")
print("Good input:", good_result)

try:
    bad_result = structured_model.invoke("Extract: the sky is blue today.")
    print("Bad input:", bad_result)
except Exception as e:
    print("Caught error:", type(e).__name__, "-", e)
```

Expected output:

```
Good input: name='Maria' age=34
Caught error: ValidationError - 1 validation error for Person...
```

This version works correctly and shows the difference clearly. It catches the broad `Exception` instead of the specific validation error class, which is fine for exploring, but not what you'd want in real code.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — catching the specific `ValidationError`

```
# structured_output_practice.py – Intermediate section
from pydantic import BaseModel, ValidationError
from langchain_openai import ChatOpenAI

class Person(BaseModel):
    name: str
    age: int

def get_structured_model() -> ChatOpenAI:
    return ChatOpenAI(model="gpt-4o-mini", temperature=0).with_structured_output(

def main() -> None:
    structured_model = get_structured_model()

    good_result = structured_model.invoke("Extract: Maria is 34 years old.")
    print(f"Good input -> {good_result!r}")

    try:
        structured_model.invoke("Extract: the sky is blue today.")
    except ValidationError as e:
        print(f"Caught a validation error, as expected: {e}")
    except Exception as e:
        print(f"Caught an unexpected error type: {type(e).__name__}: {e}")

if __name__ == "__main__":
    main()
```

Expected output:

```
Good input -> Person(name='Maria', age=34)
Caught a validation error, as expected: 1 validation error for Person...
```

Difference from Basic: catching the specific `ValidationError` (from `pydantic`) first, with a broader `Exception` fallback after it, matches Doc01's "catch the specific error you know how to handle" rule — in real code, you want to react differently to "the model's answer didn't match my shape" than to, say, a network timeout, and a single broad `except Exception` can't tell them apart.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — `.with_fallbacks(...)`, so a mismatch never raises

```
# structured_output_practice.py – Intermediate section
from pydantic import BaseModel
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

class Person(BaseModel):
    name: str
    age: int

def get_safe_model():
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    primary = model.with_structured_output(Person)
    fallback = model | StrOutputParser()
    return primary.with_fallbacks([fallback])

def describe(result) -> str:
    if isinstance(result, Person):
        return f"structured: {result!r}"
    return f"fallback (plain text): {result!r}"

def main() -> None:
    safe_model = get_safe_model()

    good_result = safe_model.invoke("Extract: Maria is 34 years old.")
    bad_result = safe_model.invoke("Extract: the sky is blue today.")

    print(describe(good_result))
    print(describe(bad_result))

if __name__ == "__main__":
    main()
```

Expected output:

```
structured: Person(name='Maria', age=34)
fallback (plain text): "The sky is blue because of the way sunlight scatters in t
```

Neither call raises. The caller has to check `isinstance(result, Person)` to know whether it got a real structured object or just fell back to plain text.

Approach 2 — retry with the validation error fed back to the model

Instead of falling back to something weaker, this approach tells the model exactly what went wrong and asks it to try again — a "self-correcting" retry, worth it when you'd rather get a real `Person` on the second try than silently accept plain text on the first failure.

```
# structured_output_practice.py – Intermediate section
from pydantic import BaseModel, ValidationError
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

class Person(BaseModel):
    name: str
    age: int

def extract_with_retry(text: str, max_attempts: int = 2) -> Person | None:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0).with_structured_output
    prompt = ChatPromptTemplate.from_template("Extract a name and age from: {text}")
    chain = prompt | model

    last_error: Exception | None = None
    for attempt in range(1, max_attempts + 1):
        try:
            return chain.invoke({"text": text})
        except ValidationError as e:
            last_error = e
            text = f"{text}\n\n(Previous attempt failed: {e}. Make sure to include"

    print(f"Gave up after {max_attempts} attempts: {last_error}")
    return None

result = extract_with_retry("The sky is blue today.")
print(result)
```

Expected output:

```
Gave up after 2 attempts: 1 validation error for Person...
None
```

On an input that's genuinely missing a name/age (like this one), retrying doesn't magically create data that was never there — this pattern is most useful for inputs the model *could* extract correctly but phrased its first answer ambiguously, not for inputs missing the required information entirely.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate detects and reports a mismatch, then stops. Approach 1 (`.with_fallbacks`) makes the chain never raise at all, trading a guaranteed `Person` for a guaranteed *some* result — at the cost of the caller needing a type check afterward. Approach 2 goes the other direction: it keeps trying to get a real `Person` , feeding the validation error back to the model as extra context, and only gives up (returning `None`) after a fixed number of attempts.

Which one should you use, and why? Intermediate's plain try/except is enough for exploring, or for a feature where a failure should stop the whole operation. Reach for Approach 1's fallback when "always return *something* usable, even if it's just plain text" matters more than guaranteeing the shape — like a chat feature where structured extraction is a nice-to-have layered on top of a normal reply. Reach for Approach 2's retry when the structured shape is actually required downstream (like Approach 2 of the Build Task below) and it's worth one extra API call to get it right, rather than silently accepting an unstructured answer or failing outright.